

Advanced Day 5 — Classes & OOP

Student Document · Advanced JS · Session 5 of 15

★ ADVANCED JS — DAY 5 OF 15

60 min

TOTAL

30 min

TEACHING

15 min

HANDS-ON

15 min

Q&A

Today's Topics

#	Topic
1	Class Syntax Basics
2	Getters and Setters
3	extends and super
4	static Methods
5	Private Fields with #
6	Custom Error Classes
7	Classes are Sugar (over Day 4)

PART 1 — FOLLOW ALONG WITH YOUR INSTRUCTOR

Topic 1

Class Syntax Basics

constructor + methods

```
class User {
  constructor(name, city) {
    this.name = name;
    this.city = city;
  }

  greet() {
    console.log(`Hi, I'm ${this.name} from ${this.city}`);
  }
}
```

// runs when you call `new User(...)`
// instance properties – own to each user

// method – lives on User.prototype (shared)

```
const a = new User("Priya", "Jaipur");
const b = new User("Aarav", "Mumbai");
a.greet(); // "Hi, I'm Priya from Jaipur"
b.greet(); // "Hi, I'm Aarav from Mumbai"

// Proof that classes ARE functions under the hood:
console.log(typeof User); // "function"
console.log(a.greet === b.greet); // true - shared via prototype
console.log(Object.getPrototypeOf(a) === User.prototype); // true (Day 4!)
```

Topic 2 Getters and Setters

[Look like properties](#) · [Run code](#)

```
class Product {
  constructor(name, priceInPaise) {
    this.name = name;
    this._priceInPaise = priceInPaise; // convention: _ means "internal, please
    don't touch"
  }

  // getter - called as p.priceInRupees (NO parentheses!)
  get priceInRupees() {
    return this._priceInPaise / 100;
  }

  // setter - called as p.priceInRupees = 50
  set priceInRupees(rupees) {
    if (rupees < 0) throw new Error("Price cannot be negative");
    this._priceInPaise = rupees * 100;
  }

  get priceWithGST() {
    return this.priceInRupees * 1.18; // 18% GST - derived, no storage
  }
}

const p = new Product("Notebook", 5000); // ₹50.00
console.log(p.priceInRupees); // 50 ← getter
console.log(p.priceWithGST); // 59 ← derived

p.priceInRupees = 100; // setter
console.log(p.priceInRupees); // 100
// p.priceInRupees = -10; // throws - validation in setter
```

[FOLLOW ALONG](#)

```
class T {
  constructor(c) { this._c = c; }
  get f() { return this._c * 9/5 +
32; }
}
const t = new T(100);
console.log(t.f);
```

→

prints?

Topic 3 extends and super

Chain set up automatically

```
class Animal {
  constructor(name) {
    this.name = name;
  }
  speak() {
    console.log(`${this.name} makes a sound`);
  }
}

class Dog extends Animal {
  constructor(name, breed) {
    super(name); // MUST call super before using `this`
    this.breed = breed; // own to Dog instances
  }

  speak() { // override
    super.speak(); // call parent's speak first (optional)
    console.log(`${this.name} barks!`);
  }
}

const d = new Dog("Bruno", "Labrador");
d.speak();
// "Bruno makes a sound" ← from Animal via super.speak()
// "Bruno barks!" ← from Dog's override

console.log(d instanceof Dog); // true
console.log(d instanceof Animal); // true – extends sets up the chain
```

⚠ **super() must come first** — Calling `this` before `super()` in a child constructor throws a `ReferenceError`. The parent has to set up `this` first.

Topic 4 static Methods

Belong to class · Not instance

```

class MathUtils {
  static gst(amount, rate = 18) {           // call as MathUtils.gst(...)
    return amount * (rate / 100);
  }

  static format(amount) {
    return `₹${amount.toFixed(2)}`;
  }
}

console.log(MathUtils.gst(1000));           // 180
console.log(MathUtils.format(1180));        // "₹1180.00"

// Static factory pattern – common
class User {
  constructor(name) { this.name = name; }

  static fromEmail(email) {                 // factory: parses an email into a User
    const name = email.split("@")[0];
    return new User(name);
  }
}

const u = User.fromEmail("priya@example.com");
console.log(u.name);                        // "priya"

// const m = new MathUtils();
// m.gst(100);                             // TypeError – static is on the
// class, not instances

```

Topic 5 Private Fields with

Language-enforced

```

class BankAccount {
  #balance;                               // declared private field
  #transactions = [];                     // can have a default

  constructor(initial) {
    this.#balance = initial;
  }

  deposit(amt) {
    this.#balance += amt;
    this.#transactions.push({ type: "deposit", amt });
  }

  withdraw(amt) {
    if (amt > this.#balance) throw new Error("Insufficient funds");
  }
}

```

```

    this.#balance -= amt;
    this.#transactions.push({ type: "withdraw", amt });
  }

  get balance() { return this.#balance; }
  get history() { return [...this.#transactions]; } // copy – don't expose internal array
}

const acc = new BankAccount(1000);
acc.deposit(500);
acc.withdraw(200);

console.log(acc.balance); // 1300
console.log(acc.history); // [{ type: "deposit", ...}, { type: "withdraw", ...}]
// console.log(acc.#balance); // SyntaxError – # is enforced by the language

```

Topic 6 Custom Error Classes

extends Error

```

class AppError extends Error {
  constructor(message, code) {
    super(message); // Error's own constructor handles message
    this.name = this.constructor.name; // sets to "ValidationError" or "NotFoundError"
    this.code = code;
  }
}

class ValidationError extends AppError {
  constructor(field, message) {
    super(message, "VALIDATION_FAILED");
    this.field = field;
  }
}

class NotFoundError extends AppError {
  constructor(resource) {
    super(`${resource} not found`, "NOT_FOUND");
  }
}

// Usage
function validateAge(age) {
  if (age < 0) throw new ValidationError("age", "Must be non-negative");
  if (age > 150) throw new ValidationError("age", "Must be under 150");
}

try {
  validateAge(-5);
} catch (e) {

```

```
if (e instanceof ValidationError) {  
  console.log(`[${e.code}] ${e.field}: ${e.message}`);  
  // "[VALIDATION_FAILED] age: Must be non-negative"  
}  
}
```

Topic 7 Classes are Sugar

Same as Day 4 underneath

```
// PRE-ES6 (yesterday)  
function Person(name) { this.name = name; }  
Person.prototype.greet = function () { console.log(this.name); };  
  
// ES6 (today)  
class Person {  
  constructor(name) { this.name = name; }  
  greet() { console.log(this.name); }  
}  
  
// Functionally identical – same prototype chain, same `new`, same `this` rules.  
// typeof Person === "function" in BOTH cases.
```

PART 2 — HANDS-ON EXERCISE (15 MIN)

Task 1 Basic Class with Getter

- Build a `Rectangle` class with `width`, `height` instance properties.
- Add a `get area()` getter that returns `width * height`.
- Add a method `scale(factor)` that multiplies both dimensions in place.
- Create a `2 × 3` rectangle, log the area, scale by `2`, log the area again.

Task 2 Inheritance with super

- Build an `Employee(name, salary)` class with a `describe()` method that logs `${name} earns ₹${salary}/month`.
- Build a `Manager(name, salary, team)` class that extends `Employee`.
- Override `describe()` to first call `super.describe()` then log `Leads team of ${team.length}`.
- Test with a Manager named "Riya", salary 80000, team `["Priya", "Aarav", "Anaya"]`.

Task 3 Private Field with `#`

- Build a `Counter` class with a `#count` private field starting at 0.
- Add `inc()`, `dec()`, and a `get value()` getter.
- Throw an `Error` if `dec()` would make the count negative.
- Create a counter, call `inc` three times, `dec` once, log the value, then try to `dec` four more times in a try/catch.

Bonus Custom Error Class

- Build a `ValidationError` class that extends `Error`. Constructor takes `field` and `message`.
- Set `this.name = "ValidationError"` and `this.field = field` inside the constructor.
- Write a `validateUser({ name, age })` function that throws a `ValidationError("name", ...)` if name is missing and `ValidationError("age", ...)` if age < 0.
- Call it twice in try/catch — once with bad name, once with bad age. Log `e.field` and `e.message`.

PART 3 — VISUAL SUMMARY

Class Anatomy

Element	Where it lives
<code>constructor</code>	Called by <code>new</code>
<code>method()</code>	Class.prototype (shared)
<code>static method()</code>	The class itself
<code>get x() / set x()</code>	Class.prototype
<code>#field</code>	Private to instance

extends / super

Code	Effect
<code>class B extends A</code>	B's prototype = A.prototype
<code>super(args)</code>	Call A's constructor
<code>super.method()</code>	Call A's method

Privacy Options

Form	Enforced by
Closure (Day 2)	Scope
<code>_x</code> underscore	Convention only
<code>#x</code> private field	The language

Quick Reference

Code	Result
<code>typeof MyClass</code>	<code>"function"</code>
<code>instance instanceof MyClass</code>	<code>true</code>
<code>MyClass.prototype.method</code>	The method
<code>Object.getPrototypeOf(instance)</code>	<code>MyClass.prototype</code>

HOMEWORK

Practice tasks before next class · Advanced Day 5

1. Build a `Vehicle(brand)` class with a `start()` method. Then build `Car(brand, doors)` and `Bike(brand)` that both extend it. Override `start()` in Car to call `super.start()` then log "Car-specific check". Test both.
2. Add a `static count` to your Counter class that tracks how many counters have been created. Increment in the constructor.
3. Build a `Temperature` class with a private `#celsius` field and getters for `celsius` and `fahrenheit`. Add a setter for `celsius` that validates `>= -273.15`.
4. Convert your Basic Day 12 `ValidationError` from try/catch into a proper class extending `Error`. Test that `instanceof ValidationError` returns true.
5. Read: javascript.info/class (full chapter) and javascript.info/private-protected-properties-methods.
6. (Bonus) Watch Akshay Saini's "Namaste JS — Class & Inheritance". ~15 minutes.